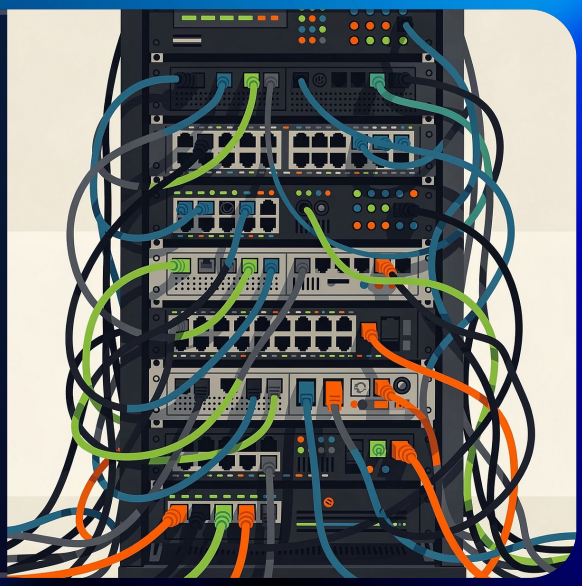
# Async Programming

Finishing **ThreadPoolExecutor** and **Queue** from last lecture, then diving into the event loop model, `async`/`await`, `asyncio`, `aiohttp`, and how async I/O compares to threading in practice.

CS / SOFTWARE ENGINEERING | ASYNC & CONCURRENCY



## 1. Learning Goals

By the end of this lecture, you should be able to confidently write and reason about asynchronous Python code. These are the concrete skills you'll walk away with:

1

### Explain the Event Loop

Describe how it differs from threading and why it's efficient for I/O-bound work

2

### Write Coroutines

Define `async def` functions and use `await` correctly inside them

3

### Run Async Code

Use `asyncio.run()` as the single entry point for any async program

4

### Concurrent Coroutines

Orchestrate multiple tasks with `asyncio.gather()` and `asyncio.create_task()`

5

### Async HTTP

Make non-blocking HTTP requests using `aiohttp` and `ClientSession`

6

### Threading vs Async

Choose the right concurrency model for a given I/O-bound problem

# 2. The Problem with Threads — Why Async Exists

Threading is a powerful tool, but it carries real costs that compound at scale. When you spin up hundreds or thousands of threads to handle concurrent I/O, the system starts to buckle.

### Why Threading Breaks Down

**High Memory Usage**

Each OS thread consumes 1–8 MB of stack space — 1,000 threads = up to 8 GB of RAM just on overhead

**Context-Switching Cost**

The OS pays a price every time it swaps which thread is running — that cost adds up fast

**Synchronization Complexity**

Race conditions, deadlocks, and lock management grow harder as thread count increases

**Poor Scaling Ceiling**

Practical limit is typically a few hundred concurrent connections before performance degrades

### The Async Solution

Async solves the scaling problem by running everything on **one thread**. Instead of blocking while waiting for a network response, the event loop suspends the current task and runs another — no thread switching, no extra memory overhead.

One async thread can efficiently manage **tens of thousands** of concurrent I/O operations — something threading simply cannot match.

# 3. The Event Loop — How Async Works

### The Core Idea

An event loop runs continuously, checking a queue of tasks. When a task hits an `await` point — a moment where it's waiting for I/O — the loop **suspends it** and runs the next ready task. When the I/O completes, the original task is resumed. No thread is ever blocked; the single thread is always doing useful work.

> **Key insight:** Most I/O time is spent waiting — not computing. Async exploits this by filling that wait time with other work.

### What Async Is NOT

Async is **not parallelism**. Only one task runs at a time on the event loop thread. It is *concurrency* — the ability to make progress on many tasks by interleaving them. For CPU-bound work (heavy computation), you still need `multiprocessing`.

### The Chef Analogy

Imagine a restaurant kitchen:

**Threading Model**

One dedicated chef per pot. Each chef blocks, stirs, and waits. Works for a small kitchen — chaotic and expensive at scale.

**Async / Event Loop Model**

One expert chef managing many pots. Puts one pot on, sets a timer, moves to the next. Never idle — always doing something useful.

> The async chef doesn't cook faster — they just **never wait**.

# 4. `async` and `await` — The Syntax

Python's async syntax is built around two keywords. Understanding their precise meaning is essential before writing any async code.

### async def — Coroutine Function

Declaring a function with `async def` makes it a **coroutine function**. Calling it doesn't execute the body — it returns a coroutine object. You must `await` it or schedule it to actually run.

```
async def fetch_data(url: str) ->
dict:
    ...  # body runs only when awaited
```

### await — Yielding Control

`await` suspends the current coroutine and gives control back to the event loop until the awaited operation finishes. It can only appear inside an `async def` function.

```
result = await fetch_data(url)
# execution resumes here after I/O
completes
```

### asyncio.run() — Entry Point

Use `asyncio.run()` exactly once at the top level of your program to start the event loop and run your root coroutine. Never nest `asyncio.run()` calls.

```
if __name__ == "__main__":
    asyncio.run(main())
```

### Never Block the Loop

Calling a blocking function (like `time.sleep()`, `open()`, or a synchronous DB call) inside async code **freezes the entire event loop**. Always use async equivalents: `asyncio.sleep()`, `aiofiles`, etc.

```
#
```

# 5. `asyncio` — The Standard Library

Python's built-in `asyncio` module provides the core primitives for running and coordinating coroutines. These four tools cover the vast majority of use cases.

### asyncio.gather()

Runs multiple coroutines **concurrently** and waits for all to complete. Returns a list of results in the same order as the inputs.

```
results = await asyncio.gather(
    fetch(url1), fetch(url2),
fetch(url3)
)
```

### asyncio.create_task()

Schedules a coroutine to run **immediately** in the background. Useful when you want to fire off work before you need the result.

```
task = asyncio.create_task(fetch(url))
# ... do other work ...
result = await task
```

### gather() + return_exceptions

By default, one failure cancels all tasks. Pass `return_exceptions=True` so failures are returned as values instead of raised — essential for fault-tolerant batch requests.

```
results = await asyncio.gather(
    *coros, return_exceptions=True
)
```

### asyncio.sleep() vs time.sleep()

`asyncio.sleep(n)` suspends the current coroutine and *yields to the event loop*. `time.sleep(n)` blocks the entire thread — never use it in async code.

```
#
```

# 6. `aiohttp` — Async HTTP Requests

Install with `pip install aiohttp`. The `aiohttp` library brings fully non-blocking HTTP to Python — a drop-in async replacement for `requests`.

### 6.1 Basic GET & ClientSession

Always use `aiohttp.ClientSession` as a context manager. Create **one session per logical operation** — never one per request, as sessions manage connection pooling and headers efficiently.

```
async with aiohttp.ClientSession() as
session:
    async with session.get(url) as
resp:
        data = await resp.json()
```

### 6.2 Multiple Concurrent Requests

Combine `ClientSession` with `asyncio.gather()` to fire many requests simultaneously. The event loop handles all the waiting — your code stays clean and sequential-looking.

```
async with aiohttp.ClientSession() as
session:
    results = await asyncio.gather(
        *[fetch(session, url) for url
in urls]
    )
```

### 6.3 Timeouts & Error Handling

Always set a timeout — never let a request hang indefinitely. Wrap requests in `try/except` to handle `aiohttp.ClientError` and `asyncio.TimeoutError` gracefully.

```
timeout =
aiohttp.ClientTimeout(total=5)
async with session.get(url,
timeout=timeout) as r:
    r.raise_for_status()
    return await r.json()
```

### 6.4 POST / PUT / DELETE & Pydantic

Session methods mirror `requests`: `session.post(url, json=payload)`, `session.put()`, `session.delete()`. Parse and validate response JSON with **Pydantic** models to catch schema mismatches early.

```
raw = await resp.json()
item = MyModel.model_validate(raw)
```

# 7. Async Patterns

Real-world async code needs more than bare coroutines. These three patterns cover the most common production scenarios: rate limiting, code reuse, and fire-and-forget background work.



### 7.1 Limiting Concurrency with Semaphore

Without a limit, `gather()` launches every coroutine at once — which can overwhelm a server or violate a rate limit. `asyncio.Semaphore(n)` acts as a gate, allowing at most `n` coroutines to run concurrently.

```
sem =
asyncio.Semaphore(10)
async def
limited_fetch(url):
    async with sem:
        return await
fetch(url)
```



### 7.2 Reusable Async Client Class

Encapsulate your session, base URL, headers, and retry logic into a class using `__aenter__` and `__aexit__`. Callers use it as an async context manager — clean, testable, and reusable across the codebase.

```
class ApiClient:
    async def
__aenter__(self):
        self.session =
aiohttp.ClientSession()
        return self
    async def
__aexit__(self, *args):
        await
self.session.close()
```



### 7.3 Background Tasks with create_task()

Use `asyncio.create_task()` for fire-and-forget work that shouldn't block your main flow — logging events, sending analytics, cache warming. Always `await` or store the task reference to avoid silent failures.

```
task =
asyncio.create_task(
 log_event(user_id,
action)
)
# task runs in the
background
```

# 8. Threading vs Async — When to Use Which

Both `ThreadPoolExecutor` and `asyncio` solve I/O-bound concurrency — but they make very different tradeoffs. Choose based on your scale, codebase, and library ecosystem.

| Aspect | ThreadPoolExecutor | asyncio + aiohttp |
|---|---|---|
| **Model** | Multiple OS threads | Single thread, event loop |
| **Memory / task** | ~1–8 MB per thread | ~few KB per coroutine |
| **Scales to** | Hundreds of connections | Tens of thousands |
| **Code style** | Familiar synchronous code | `async`/`await` everywhere |
| **Blocking libs** | Works natively | Must use async versions |

### Decision Guide

**Under 100 concurrent tasks**

Threading is simpler — less cognitive overhead, works with all existing libraries

**1,000+ tasks or async codebase**

Go async — e.g. FastAPI, aiohttp servers where the event loop is already running

**Must use blocking third-party libs**

Threading wins — you cannot easily async-ify a synchronous library

### Bridging Sync and Async

Need to call a blocking function (CPU-heavy computation, a legacy library) from inside async code? Use `loop.run_in_executor()` to offload it to a thread pool without blocking the event loop.

```
loop = asyncio.get_event_loop()
result = await loop.run_in_executor(
    None, blocking_function, arg
)
```

This is the bridge between the async world and blocking synchronous code — use it sparingly and deliberately.

# 9. Best Practices

Following these rules consistently will keep your async code correct, performant, and maintainable — in production and in your assignments.

### ✅ Always

- **Use `asyncio.run()` once**

  Call it exactly once at the program's top level as the single entry point into the event loop

- **One `ClientSession` per operation**

  Reuse sessions to benefit from connection pooling — create one per logical task, not per request

- **Set timeouts on every request**

  Use `aiohttp.ClientTimeout` to prevent hanging coroutines from stalling the entire event loop

- **Use `Semaphore` for rate limiting**

  Protect downstream servers and stay within API rate limits by capping concurrent requests

- **Pass `return_exceptions=True`**

  Prevent one failed task from cancelling all siblings in a `gather()` call

- **Validate data with Pydantic**

  Catch malformed API responses at the boundary before bad data propagates through your app

### ❌ Never

- **Call blocking functions in async code**

  `time.sleep()`, synchronous file I/O, or blocking DB calls freeze the entire event loop for every other coroutine

- **Forget `await`**

  Without `await`, you get back a coroutine object — the function body never runs, and you won't get a warning

- **Nest `asyncio.run()`**

  Running `asyncio.run()` inside a running event loop raises a `RuntimeError` — only call it once at the top level

- **Create a new session per request**

  Each `ClientSession` opens a connection pool — creating thousands of them defeats the purpose of async and wastes resources

⚠ Forgetting `await` is the single most common async bug. Always double-check that every coroutine call is awaited or scheduled as a task.

# Summary

This lecture introduced the core concepts, tools, and best practices for writing efficient, scalable asynchronous code in Python. Here are the key takeaways:

### 1 Why Async?

Optimizes I/O-bound tasks by eliminating the overhead and scalability limits of traditional multi-threading for network operations and external interactions.

### 2 Event Loop

The single-threaded heart of async, cooperatively scheduling and executing coroutines without blocking, enabling high concurrency with minimal resource usage.

### 3 Async/Await Syntax

Keywords that define and control coroutine execution, allowing explicit pauses and resumptions of I/O operations, making concurrent code readable and manageable.

### 4 Asyncio Primitives

Fundamental building blocks like Tasks, Future, and Queue for advanced concurrency control, synchronization, and inter-task communication.

### 5 Aiohttp for HTTP

The go-to library for making non-blocking HTTP requests, integrating seamlessly with the event loop for high-performance web client and server development.

### 6 Async Patterns

Essential techniques like Semaphores for rate limiting,

encapsulating logic in reusable async client classes, and `create_task()` for fire-and-forget background operations.

### 7 Threading vs. Async

Choose based on workload: threading for CPU-bound tasks or when integrating blocking libraries; async for I/O-bound scale, especially with existing async frameworks.

### 8 Best Practices

Always use `asyncio.run()` once, reuse `ClientSession`, set timeouts, use `Semaphore`, validate with Pydantic, and ensure all coroutines are `await`ed.

